

Contents

1	Rapport Chatbot Juridique — Version Étendue avec Fondements Mathématiques	2
1.1	Document mis à jour avec explications complètes des formules, choix de conception et justifications théoriques	2
1.2	Abstract	2
1.2.1	Objectives	2
1.3	1. End-to-End Pipeline Overview	2
1.4	2. Language Detection — Fondements Mathématiques et Justifications	2
1.4.1	2.1 Unicode Script Analysis (Arabic)	2
1.4.2	2.2 Statistical Language Detection (langdetect)	3
1.5	3. Intent Classification — Formules et Justifications	3
1.5.1	3.1 Pattern-Based Classification	3
1.5.2	3.2 Confidence-Based Scoring — Formule Complète	4
1.5.3	3.3 Pourquoi le bonus de 0.03 ? — Justification Détaillée	4
1.5.4	3.4 Ambiguity Detection	4
1.5.5	3.5 LLM Disambiguation	5
1.6	4. Query Routing	5
1.7	5. Query Representation (Embeddings) — Fondements Théoriques	5
1.7.1	5.1 Espace Vectoriel des Embeddings	5
1.8	6. Hybrid Re-Ranking Retrieval — Formules Détaillées	6
1.8.1	6.1 Similarité Cosinus — Pourquoi Cette Métrique ?	6
1.8.2	6.2 Score Pondéré par Source — Justification des Poids	6
1.8.3	6.3 Deduplication — Jaccard Similarity	7
1.8.4	6.4 Reranking — Deuxième Passage	7
1.9	7. Boosts Additionnels (Search Layer)	8
1.9.1	7.1 Entity Boost pour les Documents Utilisateur (document_query)	8
1.9.2	7.2 Seuils par Collection	8
1.10	8. Balanced Selection (Multi-Document)	8
1.11	9. Chunking — Paramètres et Formule	9
1.12	10. Token Estimation	9
1.13	11. Evaluation Methodology	9
1.13.1	11.1 Retrieval Metrics	9
1.13.2	11.2 BERTScore pour la Génération	10
1.13.3	11.3 Human Evaluation	10
1.14	12. Résumé des Constantes et Leur Inspiration	10
1.15	13. Références et Lectures Complémentaires	11
1.16	14. Résumé	11
1.17	15. Inspiration et Sources — D'où viennent ces concepts ?	11
1.17.1	RAG (Retrieval-Augmented Generation)	11
1.17.2	Hybrid Search (recherche hybride)	11
1.17.3	Reranking (deuxième passage)	12
1.17.4	Similarité cosinus	12
1.17.5	Déduplication Jaccard	12
1.17.6	Embeddings multilingues	12
1.17.7	BERTScore (évaluation)	12

1 Rapport Chatbot Juridique — Version Étendue avec Fondements Mathématiques

1.1 Document mis à jour avec explications complètes des formules, choix de conception et justifications théoriques

1.2 Abstract

Accessing legal and administrative regulations within academic institutions is often difficult for students and researchers. Legal texts are typically written in complex language and distributed across multiple documents. This work presents the design of a multilingual legal advisory chatbot and NLP assistant that assists users in navigating legal procedures and institutional regulations.

Example use case: A PhD student whose supervisor dies during the thesis must follow specific administrative procedures defined by university regulations. Traditional search methods require manually reading many documents. Conversational AI systems provide a more accessible solution by allowing users to ask questions in natural language. However, purely generative models may produce hallucinated legal information. Modern systems address this issue using **Retrieval-Augmented Generation (RAG)**, where responses are grounded in external documents.

1.2.1 Objectives

- Implement hybrid re-ranker retrieval over legal documents and NLP knowledge
 - Provide grounded responses using RAG
 - Evaluate system performance using retrieval and generation metrics
-

1.3 1. End-to-End Pipeline Overview

User Query → Language Detection → Intent Classification → Query Routing
→ Query Representation (Embedding) → Hybrid Retrieval → Deduplication
→ Reranking → Context Construction → LLM Generation → Final Answer

1.4 2. Language Detection — Fondements Mathématiques et Justifications

The system supports Arabic, French, and English using a **hybrid detection strategy** combining two techniques.

1.4.1 2.1 Unicode Script Analysis (Arabic)

Pattern utilisé :

`[\u0600-\u06FF\u0750-\u077F\u08A0-\u08FF\uFB50-\uFDFF\uFE70-\uFEFF]`

Formule de décision :

$$\text{lang} = \text{ar} \quad \text{si et seulement si} \quad \frac{N_{ar}}{N_{alpha}} \geq 0.3$$

Où : - N_{ar} = nombre de caractères dans les plages Unicode arabes - N_{alpha} = nombre total de caractères alphabétiques dans le texte

Pourquoi 0.3 (30%) ?

1. **Robustesse au code-mixing** : Les textes arabes modernes contiennent souvent des mots français/anglais (noms propres, termes techniques). Un seuil trop élevé (ex. 0.8) rejeterait ces textes. Un seuil trop bas (ex. 0.1) classerait incorrectement des textes latins contenant quelques caractères arabes (citations, notes).
2. **Littérature** : Les systèmes de détection de langue par script utilisent typiquement des seuils entre 0.25 et 0.4 pour l'arabe (cf. `langdetect`, `pycld2`). La valeur 0.3 offre un compromis empirique validé.
3. **Interprétation géométrique** : On définit un espace de features (N_{ar}, N_{alpha}). La règle $N_{ar} \geq 0.3 \cdot N_{alpha}$ correspond à un demi-plan : tout point au-dessus de la droite $y = 0.3x$ est classé arabe.

1.4.2 2.2 Statistical Language Detection (`langdetect`)

Si le texte n'est pas arabe, le système utilise la bibliothèque **langdetect**, qui implémente des modèles de langue basés sur des **n-grammes de caractères**.

Formule (Maximum Likelihood Estimate) :

$$L^* = \arg \max_{L \in \{\text{fr}, \text{en}, \dots\}} P(L | T) = \arg \max_L P(T | L) \cdot P(L)$$

Où : - T = le texte d'entrée - L^* = la langue prédite (meilleure estimation) - $P(L | T)$ = probabilité a posteriori de la langue sachant le texte

Pourquoi des n-grammes de caractères ?

Les n-grammes de caractères (souvent trigrammes, $n = 3$) capturent des motifs linguistiques sans dictionnaire : les séquences comme “tion”, “que”, “the” sont caractéristiques de certaines langues. Cette approche est robuste aux mots inconnus et aux textes courts.

1.5 3. Intent Classification — Formules et Justifications

Ce rapport présente uniquement les trois intents principaux du chatbot juridique. Les autres intents (platform, metadata, etc.) peuvent être ajoutés ultérieurement selon les besoins.

1.5.1 3.1 Pattern-Based Classification

Le classifieur utilise des banques de motifs regex par intention.

Intents présentés (scope du rapport) :

Intent	Purpose
legal_query	Questions sur les régulations et textes juridiques

Intent	Purpose
document_query	Questions sur les documents uploadés par l'utilisateur
general_knowledge	Questions générales (réponse directe par le LLM, sans retrieval)

1.5.2 3.2 Confidence-Based Scoring — Formule Complète

Formule de score par intention :

$$\text{score}_i = \begin{cases} 0 & \text{si } m_i = 0 \\ \min(b_i + 0.03 \cdot \min(m_i - 1, 3), 0.98) & \text{si } m_i \geq 1 \end{cases}$$

Où : - m_i = nombre de motifs qui matchent pour l'intention i - b_i = score de base (ex. 0.85 pour legal_query, 0.85 pour document_query, 0.82 pour general_knowledge)

1.5.3 3.3 Pourquoi le bonus de 0.03 ? — Justification Détaillée

Le coefficient **0.03** pour chaque motif supplémentaire est choisi pour les raisons suivantes :

1. **Rendements décroissants** : Le premier motif qui matche suffit à identifier l'intent (score de base b_i). Chaque motif supplémentaire apporte une confirmation, mais avec un gain décroissant. Un bonus trop fort (ex. 0.10) ferait qu'une requête avec 3 motifs "legal" obtiendrait $0.85 + 0.20 = 1.05$ (plafonné à 0.98), écrasant toute autre intention même si la requête est ambiguë (ex. "Est-ce légal de partager ce document ?" → legal + document).
2. **Échelle du score de base** : Les scores de base sont entre 0.82 et 0.90. Un bonus de 0.03 par motif place chaque match supplémentaire à environ 3–4% du score total. C'est assez pour départager deux intents proches (ex. legal 0.85 vs general 0.82) sans créer des écarts artificiels.
3. **Cap à 3 bonus** : On limite à $\min(m_i - 1, 3)$, donc au plus +0.09. Au-delà de 4 motifs, les matchs supplémentaires n'augmentent plus le score. Cela évite qu'une requête très chargée en mots-clés (ex. "legal law regulation droit loi") domine excessivement et masque une ambiguïté réelle.
4. **Valeur empirique** : Des valeurs de 0.02 à 0.05 sont courantes dans les classifieurs à règles. 0.03 offre un compromis : assez sensible pour refléter la multiplicité des indices, pas assez pour sur-confirmer.

Plafond à 0.98 : On évite une confiance de 1.0 car le classifieur heuristique n'est jamais certain à 100%. Une marge de 0.02 permet de distinguer "très confiant" de "parfait".

Scores de base pour les 3 intents : - legal_query (0.85) : priorité pour un chatbot juridique - document_query (0.85) : même priorité car les questions sur documents uploadés sont centrales - general_knowledge (0.82) : légèrement plus bas car les requêtes ouvertes sont plus ambiguës

1.5.4 3.4 Ambiguity Detection

Règle de détection :

$$\Delta = \text{score}_{(1)} - \text{score}_{(2)}$$

Si $\Delta < 0.15$, le classifieur considère la requête **ambiguë** :

$$\text{confidence} = \min(\text{top_score}, 0.60)$$

Pourquoi 0.15 ? Une différence de 0.15 correspond à 5 bonus de 0.03. Si deux intentions sont à moins de 0.15 l’une de l’autre, elles sont indiscernables pour un classifieur à motifs. Le plafond à 0.60 signale qu’un fallback LLM peut désambiguïser.

1.5.5 3.5 LLM Disambiguation

Quand le classifieur est incertain, un LLM reçoit la requête et la liste des intents candidats (legal_query, document_query, general_knowledge). Il retourne l’intent le plus pertinent.

1.6 4. Query Routing

Après identification de l’intent, le routeur dirige vers les sources appropriées :

Intent	Data Source
legal_query	Qdrant — collection legal_documents
document_query	Qdrant — collection document_chunks (documents de l'utilisateur)
general_knowledge	LLM directement (pas de retrieval)

1.7 5. Query Representation (Embeddings) — Fondements Théoriques

1.7.1 5.1 Espace Vectoriel des Embeddings

Chaque requête q et document d est converti en vecteur dense :

$$f : \mathcal{T} \rightarrow \mathbb{R}^{768}$$

Le modèle utilisé est **paraphrase-multilingual-mpnet-base-v2** (sentence-transformers).

Pourquoi 768 dimensions ?

- L’architecture MPNet (Microsoft) produit des vecteurs de dimension 768, un standard pour les modèles BERT-like. Cette dimension offre un bon compromis entre expressivité et coût computationnel.
- Des dimensions plus petites (384) perdent de l’information sémantique ; des dimensions plus grandes (1024+) augmentent le coût du produit scalaire sans gain proportionnel pour la similarité sémantique.

Pourquoi un modèle multilingue ?

Pour un chatbot juridique supportant arabe, français et anglais, un **seul espace vectoriel partagé** est essentiel. Les modèles multilingues (entraînés sur des corpus alignés) placent des phrases de sens équivalent dans des régions proches, quelle que soit la langue. Ainsi, une requête en arabe peut retrouver un document juridique en français si les contenus sont sémantiquement proches.

1.8 6. Hybrid Re-Ranking Retrieval — Formules Détaillées

1.8.1 6.1 Similarité Cosinus — Pourquoi Cette Métrique ?

Définition :

$$\cos(\vec{q}, \vec{d}) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \|\vec{d}\|} = \frac{\sum_{i=1}^n q_i d_i}{\sqrt{\sum q_i^2} \cdot \sqrt{\sum d_i^2}}$$

Pourquoi le cosinus et pas la distance euclidienne ?

1. **Invariance à la norme** : Le cosinus mesure l'**angle** entre les vecteurs, pas leur magnitude. Un document long et un document court exprimant la même idée auront des vecteurs de normes différentes mais de direction similaire. La similarité cosinus les traite comme équivalents.
2. **Borné dans $[-1, 1]$** : Pour des embeddings normalisés (comme ceux de sentence-transformers), le cosinus est dans $[0, 1]$, ce qui facilite l'interprétation et le seuillage.
3. **Équivalence avec le produit scalaire pour vecteurs normalisés** : Si $\|\vec{q}\| = \|\vec{d}\| = 1$, alors $\cos(\vec{q}, \vec{d}) = \vec{q} \cdot \vec{d}$. Qdrant et la plupart des bases vectorielles normalisent les vecteurs pour optimiser le calcul.

Référence : Salton & McGill (1983), *Introduction to Modern Information Retrieval* — le cosinus est la métrique standard en recherche d'information vectorielle.

1.8.2 6.2 Score Pondéré par Source — Justification des Poids

Formule :

$$s_{final}(q, d, s) = \cos(\vec{q}, \vec{d}) \times w_s$$

Poids pour les sources pertinentes au chatbot juridique :

Source	Poids w_s	Justification
Legal documents	1.05	Pour un chatbot juridique , les textes de loi et réglementations doivent être priorités. Un boost modéré de 5% les place devant les ressources génériques. Un document legal à similarité 0.80 devient 0.84.
Document chunks (utilisateur)	1.00	Les documents uploadés par l'utilisateur sont la source primaire pour document_query. Poids neutre ; la similarité sémantique suffit.

Pourquoi 1.05 pour legal ? Assez pour prioriser le juridique sans écraser les autres sources dans un merge hybride. L'inspiration vient des systèmes de recherche fédérée (Federated Search) qui utilisent des poids par source pour fusionner des résultats hétérogènes.

1.8.3 6.3 Deduplication — Jaccard Similarity

Formule :

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Où A et B sont les ensembles de mots (tokens) des deux chunks.

Règle de décision :

$$J(A, B) \geq 0.85 \Rightarrow \text{duplicate (on supprime l'un des deux)}$$

Pourquoi Jaccard ?

- **Simplicité** : Pas besoin d’embeddings pour la déduplication. Le Jaccard sur des ensembles de mots est $O(|A| + |B|)$, très rapide.
- **Robustesse aux paraphrases proches** : Deux chunks avec 85% de mots en commun sont quasi-identiques. Pour du RAG, garder les deux n’apporte pas d’information supplémentaire et gaspille le budget de contexte.

Pourquoi 0.85 ?

- 0.90 serait trop strict : des chunks avec de légères variations (numérotation, ponctuation) seraient conservés en double.
- 0.80 serait trop permissif : des chunks partiellement différents (ex. deux paragraphes d’une même section) pourraient être fusionnés à tort.
- 0.85 est un seuil standard en déduplication de documents (voir Manku et al., “Detecting Near-Duplicates for Web Crawling”).

1.8.4 6.4 Reranking — Deuxième Passage

Formule :

$$\text{score}_{\text{rerank}}(q, d) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \|\vec{d}\| + \epsilon}$$

avec $\epsilon = 10^{-9}$ pour la **stabilité numérique** (éviter une division par zéro si un vecteur est nul).

Pourquoi un second passage de similarité cosinus ?

1. **Ré-encodage** : Le reranker ré-encode la requête et les top candidats. Les embeddings peuvent différer légèrement selon le contexte (batch vs single). Un recalcul homogène assure une comparaison cohérente.
2. **Ordre final** : Après le merge pondéré et la déduplication, l’ordre peut être sous-optimal. Le reranking trie les candidats restants par similarité cosinus pure, produisant un classement plus fidèle à la pertinence sémantique.
3. **Top-K final** : Seuls les **top 5** chunks sont retournés (configurable via `TOP_K_RESULTS`). Ce nombre limite le contexte envoyé au LLM pour respecter les limites de tokens et éviter le bruit.

1.9 7. Boosts Additionnels (Search Layer)

1.9.1 7.1 Entity Boost pour les Documents Utilisateur (document_query)

Formule :

$$s = \min(1, s_{base} + 0.06 \cdot n_{entity_matches})$$

Où $n_{entity_matches}$ = nombre d’entités nommées (noms, acronymes, années) présentes à la fois dans la requête et le chunk.

Pourquoi 0.06 ?

- Les entités sont des indices secondaires ; la similarité sémantique reste primaire.
- Avec 5 entités en commun : +0.30, ce qui peut faire remonter un chunk pertinent contenant les mêmes noms propres que la requête.
- Empiriquement, des valeurs entre 0.05 et 0.08 donnent un bon équilibre (ex. “Qui est mentionné dans le document X ?”).

1.9.2 7.2 Seuils par Collection

Collection	Seuil τ	Justification
legal_documents	0.50	Les textes juridiques ont un vocabulaire spécialisé ; on accepte une similarité modérée pour des correspondances thématiques.
document_chunks (explicit doc)	0.05	Quand l’utilisateur cible explicitement un document (“explique ce PDF”), on veut retourner des chunks même pour des requêtes vagues (“résume”, “explique”). Un seuil très bas maximise le rappel.
document_chunks (broad)	0.15	Pour une recherche générale sur tous les documents, un seuil plus élevé évite le bruit.

1.10 8. Balanced Selection (Multi-Documment)

Stratégie : - **Round 1** : Sélectionner au moins **2 chunks par document** (MIN_PER_DOC = 2). - **Round 2** : Remplir les slots restants avec les chunks de score le plus élevé.

Pourquoi 2 chunks minimum par document ?

Si l’utilisateur a uploadé 3 documents et demande “Résume ces documents”, sans cette contrainte un seul document (celui avec les chunks les plus similaires) pourrait occuper les 5 slots. La stratégie garantit une **couverture** de tous les documents, reflétant l’intention de l’utilisateur.

Formulation mathématique :

$$\forall f \in \mathcal{F}, \quad |\mathcal{R}_f| \geq 2$$

où $\mathcal{F}$ est l'ensemble des fichiers et $\mathcal{R}_f$ les chunks retenus pour le fichier f .

1.11 9. Chunking — Paramètres et Formule

Paramètres : - CHUNK_SIZE = 512 tokens - CHUNK_OVERLAP = 64 tokens

Modèle conceptuel :

$$\text{chunk}_{i+1} = \text{tail}_{64}(\text{chunk}_i) \cup \text{sentences_suivantes}$$

Pourquoi 512 ?

- Les modèles d'embedding (BERT, MPNet) ont une longueur maximale typique de 512 tokens. Des chunks plus longs seraient tronqués.
- 512 tokens ≈ 2000 caractères pour l'anglais, suffisant pour un paragraphe ou une section courte.

Pourquoi 64 de chevauchement ?

- Un overlap évite de couper une phrase ou une idée en deux. Les 64 tokens de recouvrement assurent une continuité contextuelle entre chunks adjacents.
 - 64/512 ≈ 12.5% d'overlap — valeur courante dans la littérature RAG (Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”, 2020).
-

1.12 10. Token Estimation

Formule heuristique :

$$\hat{T}(x) = \max \left(1, \left\lfloor \frac{|x|}{4} \right\rfloor \right)$$

Pourquoi 4 caractères par token ?

- Pour l'anglais et le français, les tokenizers BPE (Byte-Pair Encoding) produisent en moyenne 1 token pour 4 caractères. Pour l'arabe, le ratio peut être plus faible (1:2 ou 1:3), mais la formule reste une approximation conservative.
 - C'est une heuristique rapide sans appeler le tokenizer du modèle (coûteux). Utilisée pour le budget d'historique et le déclenchement de la summarisation.
-

1.13 11. Evaluation Methodology

1.13.1 11.1 Retrieval Metrics

Metric	Formule	Interprétation
Precision@k	$P@k = \frac{\text{relevant in top } k}{k}$	Qualité du top-k. Ex. : 4 pertinents sur 5 → P@5 = 0.8.

Metric	Formule	Interprétation
Recall@k	$R@k = \frac{\text{relevant in top } k}{\text{total relevant}}$	Couverture des documents pertinents.
MRR	$MRR = \frac{1}{Q} \sum_{i=1}^Q \frac{1}{\text{rank}_i}$	Où rank_i est la position du premier document pertinent. Mesure la rapidité à trouver la bonne réponse.

1.13.2 11.2 BERTScore pour la Génération

Pourquoi BERTScore plutôt que BLEU/ROUGE ?

- BLEU et ROUGE mesurent le **recouvrement lexical** (n-grammes). Pour des réponses multilingues et des paraphrases, ils sous-estiment la qualité.
- **BERTScore** utilise des embeddings pour comparer la **similarité sémantique** entre la référence et la génération. Plus adapté aux chatbots multilingues et aux réponses qui reformulent.

Formule (simplifiée) :

$$\text{BERTScore} = \frac{1}{n} \sum_i \max_j \cos(\mathbf{r}_i, \mathbf{g}_j)$$

où $\mathbf{r}_i$ sont les embeddings des tokens de la référence et $\mathbf{g}_j$ ceux de la génération.

1.13.3 11.3 Human Evaluation

Pour un chatbot juridique, les métriques automatiques sont insuffisantes. Une évaluation humaine par des experts est recommandée sur : - **Exactitude** : information juridique correcte - **Complétude** : explication complète - **Clarté** : réponse compréhensible - **Citation des sources** : références fournies

1.14 12. Résumé des Constantes et Leur Inspiration

Constante	Valeur	Inspiration / Référence
Seuil arabe	0.30	Détection de script, compromis code-mixing
Bonus par motif	0.03	Rendements décroissants, cap à 3 (voir §3.3)
Marge d'ambiguïté	0.15	Seuil pour déclencher LLM fallback
Poids legal	1.05	Priorité textes juridiques
Seuil Jaccard	0.85	Déduplication standard (Manku et al.)
ϵ rerank	10^{-9}	Stabilité numérique
Entity boost	0.06	Signal secondaire pour entités (document_query)
Chunk size	512	Limite des modèles BERT/MPNet
Chunk overlap	64	Continuité contextuelle (~12.5%)
Top-K	5	Budget contexte LLM
MIN_PER_DOC	2	Couverture multi-document

1.15 13. Références et Lectures Complémentaires

1. **Salton, G. & McGill, M.J.** (1983). *Introduction to Modern Information Retrieval*. McGraw-Hill. — Similarité cosinus en IR.
 2. **Lewis, P. et al.** (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *NeurIPS*. — Architecture RAG.
 3. **Manku, G.S., Jain, A., & Das Sarma, A.** (2007). “Detecting Near-Duplicates for Web Crawling.” *WWW*. — Jaccard pour déduplication.
 4. **Reimers, N. & Gurevych, I.** (2019). “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.” *EMNLP*. — Embeddings pour similarité sémantique.
 5. **Zhang, T. et al.** (2019). “BERTScore: Evaluating Text Generation with BERT.” *ICLR 2020*. — Métrique d’évaluation sémantique.
 6. **Bruch, S., Gai, A., & Ingber, A.** (2022). “An Analysis of Fusion Functions for Hybrid Retrieval.” arXiv:2210.11934. — Fusion lexical/sémantique.
 7. **Nogueira, R. & Cho, K.** (2019). “Passage Re-ranking with BERT.” arXiv:1901.04085. — Reranking avec BERT.
-

1.16 14. Résumé

Ce rapport décrit les fondements mathématiques du chatbot juridique pour trois intents : **legal_query**, **document_query** et **general_knowledge**. Il couvre la détection de langue (seuil arabe 0.30), la classification d’intention (formule de score avec bonus 0.03 et cap à 3), les embeddings multilingues (768 dimensions), la similarité cosinus, les poids par source (legal 1.05), la déduplication Jaccard (0.85), le reranking, les boosts d’entités (0.06) et les seuils par collection. Les constantes sont justifiées par la littérature et l’empirisme.

1.17 15. Inspiration et Sources — D’où viennent ces concepts ?

Les choix architecturaux du système s’appuient sur des travaux académiques et des pratiques établies en recherche d’information et en NLP.

1.17.1 RAG (Retrieval-Augmented Generation)

Source principale : Lewis, P. et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *NeurIPS*.

L’idée de combiner un module de retrieval avec un générateur de langue pour réduire les hallucinations et ancrer les réponses dans des documents externes vient de ce travail. Le chatbot juridique suit cette architecture : retrieval sur documents légaux ou uploadés, puis génération par LLM à partir du contexte récupéré.

1.17.2 Hybrid Search (recherche hybride)

Sources : - **Bruch, S., Gai, A., & Ingber, A.** (2022). “An Analysis of Fusion Functions for Hybrid Retrieval.” arXiv:2210.11934 / OpenReview.

L’article montre que la combinaison de la recherche lexicale et sémantique est complémentaire pour modéliser la pertinence. Les fonctions de fusion (combinaison convexe, RRF) sont utilisées pour

fusionner des résultats de sources multiples. Notre système applique une fusion pondérée par source (legal, document_chunks) plutôt qu’une simple moyenne, en s’inspirant de cette idée de fusion.

- **Federated Search** : fusion de résultats de plusieurs collections (legal, platform, etc.) avec des poids par source — pratique courante dans les moteurs de recherche fédérés et les systèmes multi-collection.

1.17.3 Reranking (deuxième passage)

Sources : - Architecture **two-stage retrieval** : un premier passage (retriever rapide) suivi d’un second passage (reranker plus précis). Utilisée dans de nombreux systèmes IR modernes (TREC, benchmarks de deep learning).

- **Nogueira, R. & Cho, K.** (2019). “*Passage Re-ranking with BERT.*” arXiv:1901.04085 — utilisation de modèles BERT pour reranker des passages après un premier retrieval.

Notre reranker utilise une **re-encodage + similarité cosinus** plutôt qu’un cross-encoder BERT (plus coûteux pour la production), mais conserve l’idée d’un second passage pour améliorer l’ordre final.

1.17.4 Similarité cosinus

Source : Salton, G. & McGill, M.J. (1983). *Introduction to Modern Information Retrieval*. McGraw-Hill.

Le cosinus est la métrique standard en recherche vectorielle pour mesurer la similarité entre requête et document. Elle est invariante à la norme et bornée, ce qui facilite l’interprétation et le seuillage.

1.17.5 Déduplication Jaccard

Source : Manku, G.S., Jain, A., & Das Sarma, A. (2007). “*Detecting Near-Duplicates for Web Crawling.*” WWW.

Le seuil Jaccard 0.85 pour détecter les quasi-duplicats est une pratique établie pour la déduplication de documents (web crawling, chunk-level RAG).

1.17.6 Embeddings multilingues

Source : Reimers, N. & Gurevych, I. (2019). “*Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.*” EMNLP.

Sentence-BERT et les modèles dérivés (paraphrase-multilingual-mpnet-base-v2) fournissent des embeddings de phrases pour la similarité sémantique. Le choix d’un modèle **multilingue** permet un espace vectoriel partagé pour l’arabe, le français et l’anglais.

1.17.7 BERTScore (évaluation)

Source : Zhang, T. et al. (2019). “*BERTScore: Evaluating Text Generation with BERT.*” ICLR 2020.

Pour les chatbots multilingues, BERTScore est préféré à BLEU/ROUGE car il mesure la similarité sémantique plutôt que le recouvrement lexical.

Document généré à partir de l'analyse du code source (fastapi_chatbot, Plateforme/chatbot) et des rapports techniques (AI_SYSTEM_FILE_WALKTHROUGH.md, CHATBOT_SYSTEM_DESIGN_REPORT.m